

Проект. Этап № 3

Неравновесная агрегация, фрактальные кластеры

Ощепков Дмитрий, Алади Принц, Хамдамова Айжана, Козлов Всеволод

21 марта 2025

Российский университет дружбы народов, Москва, Россия

Информация

НФИбд-01-22: * Ощепков Дмитрий * Хамдамова Айжана * Козлов Всеволод * Алади Принц

Вводная часть

Существуют разнообразные физические процессы, основная черта которых — неравновесная агрегация:

- образование частиц сажи
- выращивание кристаллов соли
- распространение воды в нефти

Цель работы

Исследовать модель агрегации, ограниченной диффузией(DLA).

Задачи

- Построить модель агрегации, ограниченной диффузией
- Найти размерность, получившихся кластеров
- Построить график зависимости числа частиц в кластере от радиуса агирации

- Модель DLA, BA, CLA, CCA
- Фрактальная размерность
- График зависимости числа частиц в кластере от радиуса агирации

- Язык программирования Julia

Теоретическое описание задачи

$$d = \lim_{\epsilon \rightarrow 0} \left(\frac{\ln(N(\epsilon))}{\ln(\frac{1}{\epsilon})} \right)$$

$$\ln(N(\epsilon)) = D \ln(R) + b,$$

где D – фрактальная размерность, $N(\epsilon)$ – число частиц на расстоянии меньшем чем R , R – радиус.

Агрегация, ограниченная диффузией (diffusion-limited aggregation, DLA) — первая модель агрегации, представляющая собой шумный рост, ограниченный диффузией.

Практическая реализация

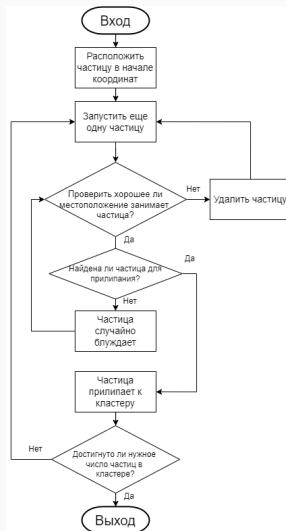


Рис. 1: Блок-схема алгоритма модели DLA

Обозначим $v^u = (0, 1)$, $v^d = (0, -1)$, $v^r = (1, 0)$, $v^l = (-1, 0)$ - шаг на 1 вверх, вниз, влево, вправо соответственно.

$\{S_n\}$ - ряд, описывающий случайное блуждание, $* = u, d, r, l$, n - количество шагов

$$S_n = \sum_{i=1}^n v_i^*,$$

$$P(v_{i+1} = v_n^*) = \frac{1}{4}$$

Результаты

DLA кластер(Diffusion-Limited Aggregation)

```
using Plots          # Библиотека для визуализации графиков
using Random         # Для работы со случайными числами
using ColorSchemes   # Цветовые схемы для визуализации

# =====
# Diffusion-Limited Aggregation (DLA)
# Моделирует процесс агрегации частиц с диффузионным ограничением
# =====

# Изменяемая структура для хранения состояния кластера
mutable struct DLACluster
    size::Int          # Размер квадратной сетки (N×N)
    target::Int         # Целевое количество частиц в кластере
    grid::Matrix{Int}  # Матрица для отслеживания позиций (0 - свободно, 1 - занято)
    particles::Vector{Tuple{Int,Int}} # Вектор координат частиц
end

# Конструктор для создания нового DLA-кластера
function DLACluster(size=100, target=500)
    # Инициализация пустой сетки
    grid = zeros{Int, size, size}

    # Начальная позиция в центре сетки
    mid = size ÷ 2      # Целочисленное деление для нахождения центра
    grid[mid, mid] = 1  # Помечаем центральную ячейку как занятую

    # Создание объекта кластера с начальными параметрами
    DLACluster(size, target, grid, [(mid, mid)])
end
```


DLA кластер(Diffusion-Limited Aggregation)

```
# Функция моделирования процесса агрегации
function simulate!(dla::DLACluster)
    # Возможные направления движения (вверх, вниз, вправо, влево)
    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    # Главный цикл: работает пока не достигнем целевого числа частиц
    while length(dla.particles) < dla.target
        # Выбираем случайную границу для старта новой частицы
        edge = rand(["top", "bottom", "left", "right"])
        # Получаем стартовую позицию на выбранной границе
        x, y = start_position(dla.size, edge)

        # Цикл движения одной частицы
        while true
            # Случайно выбираем направление движения
            dx, dy = rand(directions)
            # Обновляем координаты частицы
            x += dx
            y += dy

            # Проверяем выход за границы сетки
            if !check_bounds(dla.size, x, y)
                break # Частица ушла за пределы - прекращаем движение
            end

            # Проверяем соседство с существующим кластером
            if check_neighbors(dla.grid, x, y)
                # Фиксируем частицу в текущей позиции
                dla.grid[x, y] = 1
                # Добавляем координаты в список частиц
                push!(dla.particles, (x, y))
                break # Прекращаем движение частицы
            end
        end
    end
end
```

DLA кластер(Diffusion-Limited Aggregation)

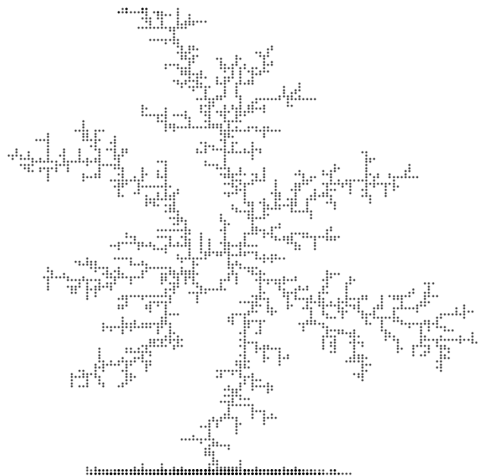
```
# Функция визуализации кластера
function plot_dla(dla::DLACluster; filename="DLA_julia.png")
    # Извлекаем координаты X и Y из списка частиц
    x = [p[1] for p in dla.particles]
    y = [p[2] for p in dla.particles]

    # Создаем точечный график
    plt = scatter(y, x,
        markersize = 1,      # Размер точек
        color = :viridis,    # Цветовая схема
        legend = false,     # Отключаем легенду
        axis = false,       # Скрываем оси
        grid = false,       # Отключаем сетку
        title = "DLA Cluster (Julia)", # Заголовок
        aspect_ratio = 1    # Сохраняем квадратное соотношение сторон
    )
    # Сохраняем график в файл
    savefig(plt, filename)
end
```

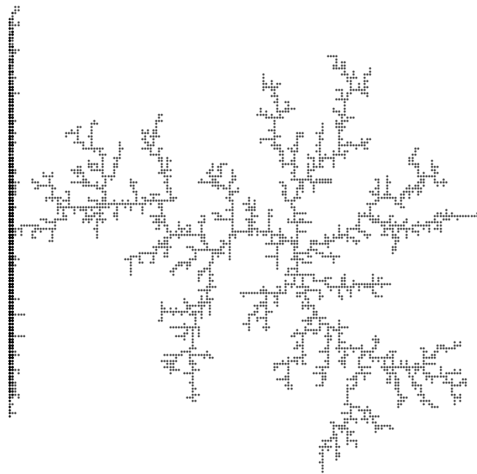
Рис. 4: DLA кластер

DLA кластер(Diffusion-Limited Aggregation)

DLA Cluster (3000 particles)



DLA Cluster (5000 particles)



BA(Ballistic Aggregation)

```
1 using Plots      # Библиотека для визуализации
2 using Random     # Генерация случайных чисел
3 using ColorSchemes # Цветовые схемы
4
5 # =====
6 # 1. Ballistic Aggregation (BA)
7 # Моделирует движение частиц по прямой к центру
8 # =====
9
10 # Структура для хранения параметров BA-кластера
11 mutable struct BACluster
12     size::Int      # Размер сетки (N x N)
13     target::Int    # Целевое количество частиц
14     grid::Matrix{Int} # Матрица для отслеживания занятых позиций (0/1)
15     particles::Vector{Tuple{Int,Int}} # Координаты частиц
16
17 # Конструктор кластера BA
18 function BACluster(size=100, target=500)
19     grid = zeros{Int, size, size} # Инициализация пустой сетки
20     mid = size ÷ 2                # Центральная позиция
21     grid[mid, mid] = 1            # Начальная частица в центре
22     BACluster(size, target, grid, [(mid, mid)]) # Создание объекта
23 end
```

Рис. 7: BA кластер

BA(Ballistic Aggregation)

```
# Функция моделирования БА
function simulate!(ba::BACluster)
    while length(ba.particles) < ba.target # Пока не достигнут целевой размер
        x, y = rand_start_position(ba.size) # Старт с границы
        prev = (x, y) # Запоминаем предыдущую позицию

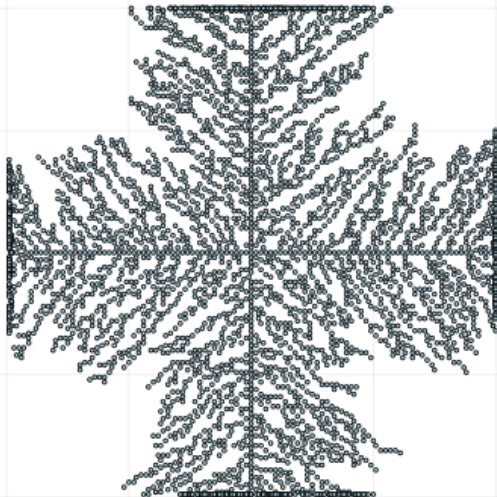
        # Движение частицы к центру
        while check_bounds(ba.size, x, y)
            # Вычисление направления к центру
            dx = sign(ba.size+2 - x)
            dy = sign(ba.size+2 - y)
            x += dx # Обновление координаты X
            y += dy # Обновление координаты Y

            # Проверка соседей
            if check_neighbors(ba.grid, x, y)
                ba.grid[prev...] = 1 # Фиксация частицы
                push!(ba.particles, prev) # Добавление в список
                break # Прекращение движения
            end
            prev = (x, y) # Обновление предыдущей позиции
        end
    end
end

# Визуализация БА
function plot_ba(ba::BACluster; filename="BA_julia.png")
    x = [p[1] for p in ba.particles] # Координаты X частиц
    y = [p[2] for p in ba.particles] # Координаты Y частиц

    plt = scatter(y, x,
        markersize = 1.5, # Размер точек
        color = :blues, # Синяя цветовая схема
        legend = false, # Без легенды
        axis = false, # Без осей
        title = "Ballistic Aggregation", # Заголовок
        aspect_ratio = 1 # Квадратное соотношение
    )
    savefig(plt, filename) # Сохранение в файл
end
```

Ballistic Aggregation



```
# =====  
# 3. Colloidal Aggregation (CLA)  
# Моделирует броуновское движение частиц  
# =====  
  
mutable struct CLACluster  
    size::Int  
    target::Int  
    grid::Matrix{Int}      # Сетка занятости  
    particles::Vector{Tuple{Int,Int}} # Список частиц  
end  
  
# Конструктор CLA  
function CLACluster(size=100, target=500)  
    grid = zeros{Int, size, size}  
    mid = size ÷ 2  
    grid[mid, mid] = 1 # Начальная частица  
    CLACluster(size, target, grid, [(mid, mid)])  
end
```

Рис. 10: CLA кластер

```
# Симуляция CLA
function simulate!(cla::CLACluster)
    directions = [(-1,0), (1,0), (0,-1), (0,1)] # Возможные направления

    while length(cla.particles) < cla.target
        # Старт частицы на границе
        edge = rand(["top", "bottom", "left", "right"])
        x, y = start_position(cla.size, edge)

        # Блуждание частицы
        while true
            dx, dy = rand(directions) # Случайный шаг
            x += dx
            y += dy

            # Выход за границы - остановка
            if !check_bounds(cla.size, x, y)
                break
            end

            # Проверка соседей
            if check_neighbors(cla.grid, x, y)
                cla.grid[x, y] = 1 # Фиксация
                push!(cla.particles, (x, y))
                break # Частица закреплена
            end
        end
    end
end
```

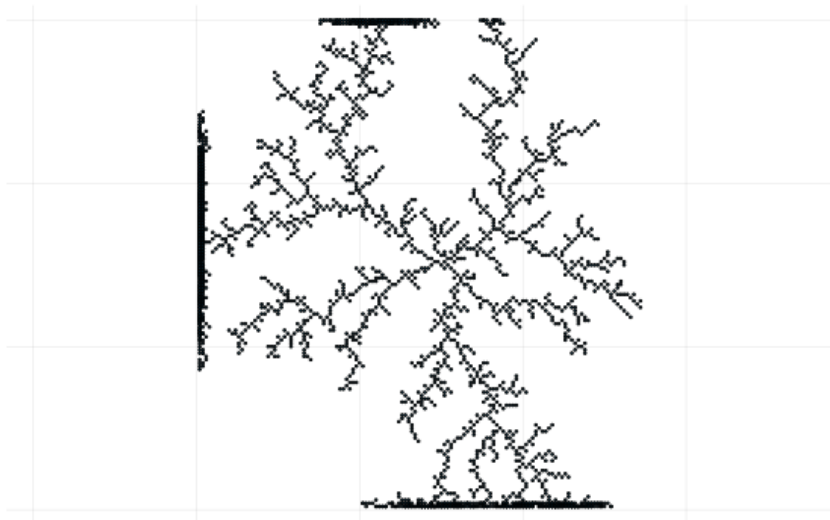


```
# Визуализация CLA
function plot_cla(cla::CLACluster; filename="CLA_julia.png")
    x = [p[1] for p in cla.particles]
    y = [p[2] for p in cla.particles]

    plt = scatter(y, x,
        markersize = 1,
        color = :thermal, # Тепловая карта
        legend = false,
        axis = false,
        title = "Colloidal Aggregation",
        aspect_ratio = 1
    )
    savefig(plt, filename)
end
```

Рис. 12: CLA кластер

Chemically Limited Aggregationcluster



CCA(Cluster-Cluster Aggregation)

```
66 # =====
67 # 2. Cluster-Cluster Aggregation (CCA)
68 # Моделирует объединение движущихся кластеров
69 # =====
70
71 # Структура для отдельного кластера
72 mutable struct ClusterCCA
73     particles::Vector{Tuple{Int,Int}} # Частицы кластера
74     position::Tuple{Float64,Float64} # Текущая позиция (X, Y)
75     velocity::Tuple{Float64,Float64} # Скорость (Vx, Vy)
76 end
77
78 # Структура для системы кластеров
79 mutable struct CCA
80     size::Int # Размер области
81     clusters::Vector{ClusterCCA} # Список кластеров
82 end
83
84 # Конструктор CCA
85 function CCA(size=100, n_clusters=20)
86     # Генерация начальных кластеров
87     clusters = [ClusterCCA(
88         [(rand(1:size), rand(1:size))], # Случайные частицы
89         (rand(1:size), rand(1:size)), # Случайная позиция
90         (randn(), randn()) # Случайная скорость
91     ) for _ in 1:n_clusters]
92     CCA(size, clusters) # Создание системы
93 end
94
95 # Основной цикл симуляции CCA
96 function simulate!(cca::CCA, steps=100)
97     for _ in 1:steps # Количество итераций
98         move_clusters!(cca) # Движение кластеров
99         merge_clusters!(cca) # Объединение кластеров
100     end
101 end
102
103 # Движение кластеров с учетом границ
104 function move_clusters!(cca::CCA)
105     for cluster in cca.clusters
106         x, y = cluster.position
107         vx, vy = cluster.velocity
108         # Обновление позиции с заворачиванием границ
109         x = mod(x + vx, cca.size)
110         y = mod(y + vy, cca.size)
111         cluster.position = (x, y)
112     end
113 end
```

CCA(Cluster-Cluster Aggregation)

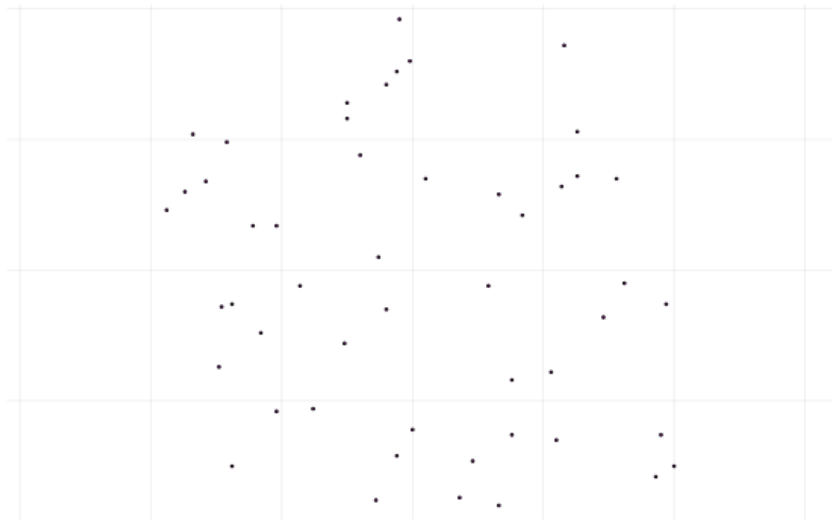
```
4
5 # Объединение близких кластеров
6 function merge_clusters!(cca::CCA)
7     merged = Set{Int}() # Множество объединенных кластеров
8     new_clusters = ClusterCCA[] # Новый список кластеров
9
10    for (i, c1) in enumerate(cca.clusters)
11        i in merged && continue # Пропуск объединенных
12
13        # Проверка соседних кластеров
14        for j in i+1:length(cca.clusters)
15            c2 = cca.clusters[j]
16            dist = hypot(c1.position[1]-c2.position[1], c1.position[2]-c2.position[2])
17
18            # Условие объединения
19            if dist < 3.0
20                push!(merged, i, j) # Добавление в объединенные
21                # Создание нового кластера
22                new_particles = vcat(c1.particles, c2.particles)
23                push!(new_clusters, ClusterCCA(
24                    new_particles,
25                    ((c1.position[1]+c2.position[1])/2, # Средняя позиция
26                    ((c1.position[2]+c2.position[2])/2),
27                    ((c1.velocity[1]+c2.velocity[1])/2, # Средняя скорость
28                    ((c1.velocity[2]+c2.velocity[2])/2)
29                ))
30            end
31        end
32        # Добавление необъединенных кластеров
33        i ∉ merged && push!(new_clusters, c1)
34    end
35    cca.clusters = new_clusters # Обновление списка
36 end
```

```
# Визуализация CCA
function plot_cca(cca::CCA; filename="CCA_julia.png")
    plt = plot(legend=false, axis=false, aspect_ratio=1)
    colors = ColorSchemes.rainbow # Радужная палитра

    # Отрисовка каждого кластера
    for (i, cluster) in enumerate(cca.clusters)
        x = [p[1] for p in cluster.particles]
        y = [p[2] for p in cluster.particles]
        scatter!(plt, y, x,
            markersize=1.2,
            color=colors[(i % 10)+1] # Циклический выбор цвета
        )
    end
    savefig(plt, filename)
end
```

Рис. 16: CCA кластер

Cluster-Cluster Aggregation



При построении 17 моделей с ограничением по радиусу от 130 до 290 получили $D = 1.717$.

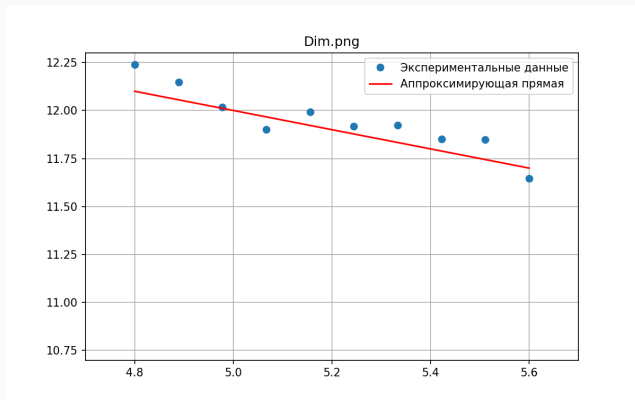


Рис. 18: График зависимости числа частиц в кластере от радиуса гирации

Заключение

- Построены модели DLA, BA, CLA, CCA
- Найдена фрактальная размерность, получившихся кластеров DLA
- Построен график зависимости числа частиц в кластере от радиуса гирации DLA

1. Медведев Д.А. и др. Моделирование физических процессов и явлений на ПК: Учеб. пособие. Новосибирск: Новосиб. гос. ун-т, 2010. 101 с.
2. Sander L.M. Diffusion-limited aggregation: A kinetic critical phenomenon? Contemporary Physics, 2000.